FreeRTOS / Hafta 03

Queue, bekleme ve yük altında veri akışı

Bu rehber, bu haftanın dersini dinlemeden ödeve hazırlanabilmen için yazıldı. Önce bir mesajın bellekteki yolculuğunu, sonra task'ların zaman içindeki davranışını, en son da bunların nasıl ölçüleceğini öğreneceksin.

Temel soru: Üretici veri hazırlıyor ama tüketici aynı hızda yetişemiyorsa sistem ne yapmalı? Daha çok bellek ayırmak, üreticiyi bekletmek ve mesajı reddetmek farklı bedeller taşır. Bu haftanın hedefi bu bedelleri kod ve iz kaydıyla açıklamak.

Öğrenme rotan

• Bölüm 1-4: Task durumları, mesaj kopyası ve queue API'leri.

• Bölüm 5-7: Timeout, bekleme listeleri ve yoğun yükün nedenleri.

• Bölüm 8-10: A/B/C deneyi, kaynak kodun okunması ve ölçümler.

• Bölüm 11-12: SystemView kanıtı ve örnek sonuçların yorumlanması.

• Bölüm 13-14: Cevaplı alıştırmalar ve ödeve geçiş.

Bu rehberi nasıl çalışmalısın?

Her bölümün sonunda dur ve örneği kendi cümlelerinle anlat. Kod gördüğünde yalnız fonksiyon adını öğrenme: çağrı öncesinde queue dolu mu, task hangi durumda, dönüşten sonra neyi biliyoruz sorularını sor. Çalışma süresi kişiye göre değişir; iki oturuma bölmek uygundur.

Gerekli önbilgi: C struct ve pointer kavramı, sonsuz task döngüsü, task önceliği ve milisaniye birimi. Önceki haftalardaki ayrıntıları hatırlamıyorsan ikinci sayfa zamanlama temelini tekrar kurar.

Kaynak ve kapsam

Ana kaynak, verilen Hafta_3_Ders_Paketi.zip içindeki ders/ödev slaytları, referans firmware ve kurulum rehberidir. Açıklamalar bu malzemelerden geliştirilmiştir. Bu belge dersin konuşma dökümü değildir; bağımsız çalışma rehberidir. Kaynaklar son sayfadadır.

Bu çalışmada kartta yeni ölçüm alınmadı. Rehberdeki sayısal öğretim örnekleri varsayımsal olarak, paketten alınan gerçek kayıtlar ise referans kayıt olarak etiketlenir. Ödevin ayrıntılı gereksinimleri aynı klasördeki spec.md dosyasındadır.

01 / Task neden çalışmıyor?

Ready, Running ve Blocked

Tek çekirdekte aynı anda bir task CPU üzerinde komut yürütür. Scheduler, çalışabilecek task'lar arasından önceliğe göre seçim yapar. Bu pakette yüksek sayı daha yüksek önceliktir: Console P3, Producer P2, Consumer P1.

Durum	Anlamı	Queue örneği
Ready	Çalışabilir; CPU sırasını bekler	Veri geldi, tüketici uyandırıldı
Running	Şu anda CPU üzerindedir	Mesaj alanlarını hesaplıyor
Blocked	Bir olay veya süre bekler	Boş queue'da veri bekliyor

Blocked task CPU'da boş bir döngü döndürmez. Scheduler onu seçilebilir task'lar arasından geçici olarak çıkarır. Bu yüzden yüksek öncelikli bir üreticinin bloklanması, düşük öncelikli tüketicinin çalışmasını sağlayabilir.

Küçük olay dizisi

Producer P2 dolu kuyruğa 20 ms beklemeyle send çağırır. Yer olmadığı için Blocked olur. Başka daha yüksek Ready task yoksa Consumer P1 çalışır ve bir mesaj alır. Yer açılması Producer'ı Ready yapabilir. Producer tekrar CPU aldığında send işlemine devam eder.

Buradaki iki farklı soruyu ayır: Mesaj hangi sırayla alınacak? Buna kuyruk sırası cevap verir. Hangi task CPU alacak? Buna scheduler cevap verir. Mesajı öne koymak, onu gönderen task'ın önceliğini artırmaz.

Beklemek ile işlem yapmak farklıdır

`vTaskDelay(pdMS_TO_TICKS(20))`, task'ın zaman beklemesine neden olur. Bu süre boyunca başka task'lar çalışabilir. Referans tüketici bu çağrıyla dış bir servisin tamamlanmasını modeller. Bu, CPU üzerinde 20 ms hesap yapmakla aynı deney değildir.

Hazır olmak hemen çalışmak değildir

Bir olay task'ı Ready yaptığında CPU'yu hemen alması garanti değildir. Daha yüksek öncelikli hazır task'lar veya kesmeler geciktirebilir. Bu nedenle bir timeout değeri ile ölçülen duvar saati süresi birebir aynı olmak zorunda değildir.

Kendini kontrol et: Consumer düşük öncelikli diye hiç çalışamaz mı? Hayır. Producer bloklandığında veya gecikme beklediğinde Consumer çalışabilir. Ancak sürekli Ready kalan daha yüksek öncelikli task onun ilerlemesini engelleyebilir.

02 / Queue ne saklar?

Bir mesajın üç ayrı kopyası

Queue, eleman boyutu ve kapasitesi oluşturulurken belirlenen sınırlı bir depodur. Üretici ve tüketiciyi zaman bakımından birbirinden ayırır: üretici gönderdiğinde tüketicinin o anda hazır olması gerekmez, yeter ki depoda yer olsun.

```
typedef struct {
    uint32_t sequence;
    uint32_t generated_us;
    uint32_t value;
} Message;

QueueHandle_t q = xQueueCreate(4, sizeof(Message));
configASSERT(q != NULL);
```

Bu hedefte Message 12 bayttır. Dört mesajın payload alanı 48 bayt, on altı mesajınki 192 bayttır. Bunlar yalnız veri alanlarıdır; queue yönetimi ve dinamik bellek ayırmanın ek maliyeti de vardır. Genel kodda elle 12 yazmak yerine sizeof kullanılır.

Kopyalama örneği

Producer'ın yerel `msg.value` alanı 42 olsun. Başarılı send, struct'ın baytlarını queue'nun alanına kopyalar. Producer daha sonra kendi `msg.value` alanını 99 yapsa da queue'daki kopya 42 kalır. Consumer receive yaptığında değer bu kez consumer'ın kendi Message nesnesine kopyalanır ve kuyruktaki eleman çıkarılır.

Producer nesnesi -> queue içindeki payload kopyası -> Consumer nesnesi

Gönderilen `&msg`, kopyalanacak verinin başlangıç adresidir. API'ye adres vermek, queue'nun pointer sakladığı anlamına gelmez. Saklanacak bayt sayısını xQueueCreate içindeki item size belirler.

Pointer kuyruğunda fark

Eleman boyutunu `sizeof(Message *)` seçersen yalnız adres kopyalanır; adresin gösterdiği Message kopyalanmaz. Alıcı okurken nesne hâlâ geçerli mi, üretici onu değiştirdi mi sorularını uygulamanın çözmesi gerekir. Bu haftanın deneyinde struct değer olarak taşınır; pointer sahipliği ayrı bir zorunlu çalışma değildir.

Başarı neyi kanıtlar?

Send'in başarılı olması, mesajın queue'ya kabul edildiğini kanıtlar. Tüketicinin onu aldığını veya işlemini tamamladığını kanıtlamaz. Bu üç anı ayrı tutmak daha sonra accepted, received ve consumed sayaçlarını doğru okumayı sağlar.

03 / Queue işlemleri

Ön, arka, okuma ve göz atma

Kuyruğun front ucu bir sonraki okumayı belirler. Back ucu yeni mesajların normalde eklendiği taraftır. Yalnız arkaya gönderim kullanılırsa ilk giren ilk çıkar: FIFO.

API	Başarılı olursa	Doluluk etkisi
xQueueSend / xQueueSendToBack	Arkaya kopyalar	+1
xQueueSendToFront	Öne kopyalar	+1
xQueueReceive	Öndekini alır ve çıkarır	-1
xQueuePeek	Öndekini kopyalar, bırakır	0

Adım adım sıra örneği

Aşağıdaki çizim mantıksal sıradır; soldaki eleman front'tur. Kapasite 4 ve tüm işlemlerde bekleme 0 olsun.

```
Başlangıç          [ boş ]
Back(10)           [10]
Back(20)           [10, 20]
Front(99)          [99, 10, 20]
Peek -> 99         [99, 10, 20]
Receive -> 99      [10, 20]
Front(88)          [88, 10, 20]
Receive -> 88      [10, 20]
```

Peek aynı elemanı tekrar tekrar gösterebilir; onu tüketmiş olmazsın. Front ise kuyruk doluyken eski elemanı otomatik silmez. Bekleme 0 ise başarısız döner. Front kullanımı kapasite kuralını değiştirmez.

Öne eklemenin bedeli

Sürekli yeni mesajları öne koyarsan arkadaki eski mesajlar uzun süre bekleyebilir. Bu, genel bir sayısal mesaj önceliği sistemi değildir. Önce 99 sonra 88 öne eklenirse 88 daha önce okunur; ikisi de eski mesajların önüne geçer.

Kernel bu mantıksal sırayı gerçekleştirmek için her işlemde bütün mesajları RAM'de kaydırmak zorunda değildir. Queue depolamasını dairesel alan ve konum bilgileriyle yönetir. Öğretim çizimindeki kutuların kayması fiziksel bellek kopyasının birebir gösterimi değildir.

Ödevde A/B/C karşılaştırmaları `xQueueSend`, yani arkaya gönderim kullanır. Front denemesini kavram öğrenmek için yap; ana deneye ekleyip kapasite ve timeout karşılaştırmasını bozma.

04 / Send ve Receive nasıl okunur?

Dönüş değerini atlamak veri kaybını gizler

```
Message msg = { .sequence = 1, .value = 42 };
BaseType_t r = xQueueSend(q, &msg, 0);
if (r == pdPASS) {
    accepted++;
} else {
    failed++;
}
```

İlk argüman hangi queue'yu kullanacağını, ikincisi kopyalanacak nesnenin adresini, üçüncüsü yer yoksa ne kadar beklenebileceğini söyler. Queue'da yer varsa bekleme parametresi pozitif olsa bile o kadar süreyi doldurmak için beklenmez.

Bekleme 0 ve queue doluysa çağrı yer açılmasını beklemeden başarısız döner. Uygulama ister sayıp bırakır, ister ayrı bir politika uygular. Referans başarısızlığı sayar ve aynı mesajı yeniden denemez. Bu yüzden failed, burada teslim edilemeyen girişimleri görünür kılar.

```
Message out;
if (xQueueReceive(q, &out, pdMS_TO_TICKS(25)) == pdPASS) {
    // out artık geçerli alınan mesajı içerir.
    ProcessMessage(&out);
} else {
    // Veri alınmadı; out'u yeni mesaj diye işleme.
}
```

ProcessMessage burada öğretim amaçlı bir isimdir; referans firmware'deki gerçek tüketici alınan zamanı kaydeder ve 20 ms servis beklemesi yapar. Receive için hedef nesne, queue item size kadar veriyi alabilecek boyutta olmalıdır.

Boş kuyruğu sürekli sormak

Receive'i 0 beklemeyle çok sık döngüde çağırmak polling yapar. Yüksek öncelikli tüketici bu şekilde sürekli Ready kalırsa üreticinin çalışmasına zarar verebilir. Pozitif bekleme, veri gelene kadar task'ın Blocked olmasını sağlar. Timeout yolu hata olmak zorunda değildir; uygulamanın normal akışının bir parçası olabilir.

Kontrol edip sonra kullanmak

`uxQueueMessagesWaiting(q)` anlık doluluğu verir. Bu değeri okuyup daha sonra send yapana kadar başka bir task queue'yu değiştirebilir. Doluluk sorgusu teşhis için yararlıdır; send/receive dönüş değerinin yerine geçmez.

05 / Timeout kesin süre değildir

Tick, bloklanma ve duvar saati

Queue bekleme argümanı tick cinsindedir. Bu pakette tick 1 kHz olduğundan bir tick nominal olarak 1 ms'dir. Başka projelerde bunu varsayma; `pdMS_TO_TICKS(20)` kullanarak milisaniyeyi RTOS zaman birimine çevir. Dönüşüm ve tick fazı çözünürlüğü etkiler.

Dolu queue, 20 ms bekleme

Producer send çağırdığında yer yoksa kernel onu bekletir. Consumer süre dolmadan bir mesaj alırsa yer açılır ve bekleyen gönderici Ready yapılabilir. Producer CPU aldığında koşulu yeniden değerlendirip ilerler. Yer hiç açılmazsa timeout yolu başarısızlıkla sonuçlanabilir. Pozitif bekleme, başarı garantisi değildir.

Sayısal zaman örneği - öğretim modeli

Send çağrısı t=0'da başlasın. Consumer t=7 ms'de yer açsın. Producer t=9 ms'de CPU alıp gönderimi tamamlasın. Çağrı çevresinde ölçülen süre yaklaşık 9 ms olur; timeout 20 ms diye çağrının mutlaka 20 ms sürmesi beklenmez.

Tersine timeout süresi dolduğu hâlde task CPU alamıyorsa fonksiyondan dönüş daha sonra gözlenebilir. API'nin bekleme bütçesini, çağrının duvar saatindeki mutlak üst sınırı gibi yorumlama. Preemption ve ölçüm işaretleri de süreye girebilir.

Bloklanma ile busy-wait farkı

Blocked üretici yer açılmasını beklerken komut yürütmez. Consumer çalışıp kuyruğu boşaltabilir. Üretici boş bir while döngüsünde tekrar tekrar deneseydi CPU'yu kullanmaya devam ederdi. Özellikle Producer önceliği yüksekken bu iki yaklaşım aynı sonuç vermez.

Kritik bölge tuzağı

```
// Yanlış tasarım: bekleme gerekebilecek çağrı
taskENTER_CRITICAL();
xQueueSend(q, &msg, pdMS_TO_TICKS(20));
taskEXIT_CRITICAL();

// Bu deney için doğru yer: normal task bağlamı
BaseType_t r = xQueueSend(q, &msg, pdMS_TO_TICKS(20));
```

Queue kendi iç senkronizasyonunu sağlar. Başka task'ın ilerlemesine ihtiyaç duyabilecek bir çağrıyı kritik bölge içinde bekletme. Önceki haftanın 0 tick kullanan kodunu yalnız timeout'u değiştirerek taşımak bu hataya yol açabilir. ISR içinden ise normal task API'si yerine uygun FromISR sürümü gerekir; ISR bekleyemez.

06 / Queue'nun içindeki üç bilgi

Mesaj depolamak ile task bekletmek farklıdır

Queue yalnız payload dizisi değildir. Kernel ayrıca kaç mesaj bulunduğunu ve kimlerin bir olay beklediğini izler. Aşağıdaki isimler kaynak kodu okurken yardımcı olur; uygulamanın doğrudan değiştireceği alanlar değildir.

İç alan	Neyi temsil eder?
uxMessagesWaiting	İçerideki mesaj sayısı
xTasksWaitingToSend	Yer açılmasını bekleyen göndericiler
xTasksWaitingToReceive	Mesaj gelmesini bekleyen alıcılar

Doluluk mesaj sayısıdır; bayt sayısı ya da bekleyen task sayısı değildir. Bekleme listeleri mesaj kopyalarını değil task'ların event-list düğümlerini bağlar. Bir task'ın tüm TCB'si queue payload alanına kopyalanmaz.

Dolu queue senaryosu

Kapasite 3, doluluk 3 olsun. P3 ve P1 göndericileri pozitif timeout ile yer bekliyor. Consumer bir mesaj aldığında doluluk 2 olur. En yüksek öncelikli bekleyen gönderici Ready yapılır. Onun tekrar Running olması ve send'i tamamlaması ayrıca gerçekleşir. Açılan slot o task'a kalıcı olarak rezerve edilmiş sayılmaz.

Boş queue senaryosu

Doluluk 0, iki alıcı mesaj bekliyor olsun. Başarılı bir send doluluğu artırır ve en yüksek öncelikli bekleyen alıcıyı uyandırabilir. Mesajların FIFO sırası ile alıcıların öncelik sırası ayrı mekanizmalardır. Alıcı çalışıp receive yaptığında doluluk azalır.

Timeout'lu bir bekleyici ayrıca kernel'in zaman takibine dahildir. Timeout dolduğunda event bekleme listesinden de çıkarılır ve yeniden çalışabilir hâle gelir. Olayla uyanma ile süre dolması, aynı API çağrısının farklı ilerleme yollarıdır.

Neyi gözlemleyebilirsin?

Trace'te Producer'ın Running'den Blocked'a, ardından Ready/Running'e geçişi ve Consumer'ın arada receive yapması, yer bekleme açıklamasını destekler. Sadece task'ın görünmediği bir boşluk, hangi nedenle bloklandığını tek başına kanıtlamaz; API olayıyla birlikte oku.

Kendini kontrol et: Kuyrukta 3 mesaj varsa 3 task mı bekliyordur? Hayır. Depolanan mesaj sayısı ile bekleyen task sayısı birbirinden bağımsız kavramlardır.

07 / Kuyruk neden dolar?

Ortalama hız ve kısa süreli yığılma

Bir depo, giriş ile çıkış arasındaki geçici farkı saklar. Uzun vadede üretim tüketimden hızlıysa sonlu bir queue bu farkı sonsuza kadar saklayamaz. Queue büyütmek işlem hızını kendiliğinden artırmaz.

Sürekli yük - öğretim hesabı

Üretici saniyede 100 mesaj üretiyor, tüketici saniyede 50 mesaj işleyebiliyorsa saniyede yaklaşık 50 mesaj birikir. Başlangıçta boş 16 elemanlık depo, ideal sürekli hız modelinde yaklaşık 16/50 = 0,32 saniyelik farkı karşılar. Gerçek dolma anı zamanlama ve mesajların geliş biçimine bağlıdır; bu bir donanım ölçümü değildir.

Burst: ortalama düşük olsa da taşma olabilir

Normal referans yükünde girişimler arasında 100 ms, tüketicinin servis beklemesinde 20 ms vardır. Bu aralıklar tüketicinin yetişmesine imkân verir. Burst bölümünde ise 24 girişim art arda yapılır. Producer P2, Consumer P1'den yüksek önceliklidir; 0 tick send'lerle çalışmaya devam ederken tüketici araya giremeyebilir. Queue hızla dolar.

Üç olası politika

• Az kapasite, bekleme yok: Üretici hızlı ilerler; dolu durumda girişimler reddedilir.

• Daha büyük kapasite: Daha fazla mesaj tutulabilir; RAM maliyeti ve bekleyenlerin gecikmesi artabilir.

• Doluysa bekle: Üretici yavaşlar, tüketici ilerleme fırsatı bulur; üreticinin diğer işleri gecikebilir.

Üreticiye geri yansıyan bu yavaşlama backpressure olarak adlandırılır. Amaç yalnız failed sayısını küçültmek değildir. Sensörün bir sonraki örneğini zamanında üretmesi gerekiyorsa uzun send beklemesi başka bir gereksinimi bozabilir.

Gecikme sezgisi

Bir mesajın önünde k servis işi varsa ve her servis yaklaşık 20 ms sürüyorsa bekleme kabaca k x 20 ms mertebesinde olabilir. Dört iş için 80 ms, on altı iş için 320 ms, yalnız konum varsayımını anlatan sayılardır. Scheduler etkisini, devam eden servisin kalanını ve enqueue öncesi beklemeyi ayrıca düşünmelisin.

Kapasiteye bakarak her mesajın gecikmesini veya deney maksimumunu doğrudan hesaplayamazsın. Kuyruktaki gerçek konum ve task'ların ne zaman çalıştığı belirleyicidir.

08 / A, B ve C tam olarak nedir?

Deneyi tek değişkenle kurmak

Varyant	Mesaj kapasitesi	Send beklemesi
A	4	0 tick
B	16	0 tick
C	4	pdMS_TO_TICKS(20)

A ile B arasında yalnız kapasite, A ile C arasında yalnız bekleme politikası değişir. B ile C'yi doğrudan kıyaslayarak farkı tek bir nedene bağlayamazsın: iki ayar da farklıdır. Bu haftanın isimleri önceki haftanın öncelik deneylerini temsil etmez.

Sabitler: Producer P2, Consumer P1, Console P3; 12 bayt Message; 20 ms modellenmiş servis beklemesi. Normal yükte 44 girişimin her birinden sonra 100 ms ara vardır. Yoğun yükte 10 normal, 24 art arda, 10 normal girişim yapılır. Toplam yine 44'tür.

Ölçmeden önce tahmin et

A/yoğun için küçük queue'nun dolmasını ve bazı girişimlerin başarısız olmasını bekleriz. B/yoğun daha çok mesaj kabul edebilir ama bazı mesajlar daha uzun bekleyebilir. C/yoğun üreticiyi yer açılana kadar bekletebilir; bu kabulü iyileştirebilir, send süresini artırabilir. Bunlar sınanacak tahminlerdir, sonuç garantisi değildir.

C'de 24 girişimin duvar saatindeki süresi uzayabilir. Deney aynı sayıda girişim ister; hepsinin aynı fiziksel zaman damgasında başlamasını istemez. Üretici zamanlamasındaki bu değişiklik bekleme politikasının sonucudur.

Yedi koşunun amacı

Üç varyantın her birini normal ve yoğun yükte çalıştırmak altı koşu eder. Yedinci koşu A/yoğun için trace kapalıdır; A/yoğun trace açık koşusuyla karşılaştırılır. Böylece ölçüm aracının kayda bağlı etkisini de tartışırsın.

Trace OFF, zaman sayaçlarını ve CSV üretimini kapatmaz. Firmware'den bütün trace kodunu derleme sırasında çıkarmakla da aynı şey değildir. Bu deneyden yalnız kayıt etkinliğinin etkisi hakkında çıkarım yap.

Önce bir normal koşuyla kayıt zincirini öğren. Asıl karşılaştırmada ayarları sabitle, her koşuya kimlik ver ve önceki iz dosyasını kaydetmeden yeni koşuya başlama.

09 / Referans kodu okuma rehberi

app.c içinde veri yolunu takip et

Başvuru dosyası: `kaynak/Hafta_3/firmware/src/app.c`. Önce bütün dosyayı ezberlemek yerine aşağıdaki beş parçayı sırayla incele.

1. app_init: kaynaklar ve roller

q4 ve q16 queue'ları başlangıçta birlikte oluşturulur. Koşu sırasında seçilen handle message_q olur. Bu nedenle B seçmek, çalışma anında tek başına RAM tahsisini 4'ten 16'ya değiştirmek değildir; referans her iki kuyruğa zaten sahiptir. Ayrıca seri komut byte'ları için ayrı rx_q vardır. Ödevde incelenen veri yolu message_q'dur.

2. command: deney ayarını seçmek

RUN komutu varyant ve yükü seçer. B için q16, diğerleri için q4 kullanılır. C için 20 ms bekleme, diğerleri için 0 seçilir. Queue ve sayaçlar sıfırlanır; consumer ve producer task bildirimleriyle başlatılır. Bildirimler burada deney başlangıcını yönetir; dersin ana veri aktarımı queue'dadır.

3. Producer: bir mesaj oluşturmak

Her girişimde sequence 1'den 44'e ilerler, generated_us alınır, attempted artırılır. SEND_BEGIN işareti ve send çevresindeki süre kaydı kullanılır. Sonuç accepted veya failed sayacını artırır. Yoğun bölüm mesaj 11-34'tür; bu grubun içindeki girişimlerin arasında normal 100 ms aralar bulunmaz.

4. Consumer: almak ve tamamlamak

Başarılı receive sonrası latency kaydedilir ve received işaretlenir. Sonra 20 ms vTaskDelay ile servis beklenir; consumed artırılır ve DONE olayı üretilir. Dolayısıyla alınma zamanı ile tamamlanma zamanı farklıdır. Queue'dan çıkarılmış ama servisi süren bir mesaj olabilir.

5. Console: bitiş ve dışarı aktarım

producer_done ve consumer_done gerçekleşince sonuçlar seri porta yazılır. Bu, ölçüm esnasında her mesaja printf eklemenin davranışı değiştirmesini önler. UART burada veri yolunun servis hızını ölçmez; deney komutu ve sonuç dosyalarını taşır.

Kod okumada kendine sor

`generated_us` send'den önce mi sonra mı alınıyor? Önce. C'de queue'ya girmeden beklenen süre latency içinde mi? Evet. consumed receive'ten hemen sonra mı artıyor? Hayır, servis beklemesinden sonra. Bu üç cevap, ölçümün ne söylediğini belirler.

10 / Ölçümleri doğru adlandırmak

İki zaman aralığı, üç tamamlanma seviyesi

Alan	Ne demek?
attempted	Kaç gönderme girişimi yapıldı?
accepted / failed	Kaçı kabul edildi / başarısız oldu?
received	Bu mesaj tüketici tarafından alındı mı?
consumed	Kaç mesajın modellenmiş servisi tamamlandı?

Koşu sonunda attempted = accepted + failed olmalıdır. Queue boşaltılıp son servis bittikten sonra accepted = consumed beklenir. Henüz servis süren bir anda bu eşitliği aramak hatalıdır.

send_call_us

Gönderme çevresindeki bitiş zamanı eksi başlangıç zamanıdır. Task'ın bloklanması, CPU'dan alınması ve yakındaki trace işaretinin maliyeti süreye girebilir. Saf CPU yürütme süresi diye adlandırılmaz. Başarısız send de zaman harcayabilir; özellikle timeout ile.

generation_to_receive_us

Receive anı eksi mesajın üretim zamanıdır. C'de queue'ya kabul edilmeden önceki yer beklemesi de buna dahildir. Saf kuyruk içinde kalış süresi değildir. Receive sonrasındaki 20 ms servis beklemesi de bu ölçüme dahil değildir; bu yüzden uçtan uca tamamlanma süresi diye sunulmaz.

Ortalama örneği - varsayımsal

Dört girişimin üçü alındı ve gecikmeleri 10, 20, 60 ms olsun. Dördüncüsü başarısızsa gecikme ortalaması (10+20+60)/3 = 30 ms'dir; maksimum 60 ms, başarısızlık oranı %25'tir. Başarısız mesaja 0 yazarak dörde bölmek 22,5 ms üretir ve gerçekte olmayan bir iyileşme gösterir.

CSV'de alınmayan mesajın gecikmesi boş bırakılır. Ham seri satırlarında başlangıçtan kalan 0 bulunabilir; received bayrağına bakmadan onu ölçülmüş sıfır gecikme sayma.

Analiz tablosunu nasıl kurmalısın?

Her koşu için dört sayacı, başarılıların send ve receive-gecikmesi ortalama/maksimumlarını yaz. Hangi satırları kullandığını ve örnek sayısını belirt. Bütün girişimlerin send sürelerini ayrıca özetlemek timeout maliyetini gösterir. Sonuç bulunmayan gruba N/A yaz.

Gözlenen maksimum yalnız bu kayıtta görülen en büyük değerdir. Bütün olası yükler için kesin üst sınır kanıtı değildir. Tekrarlar varyasyonu görmene yardımcı olur; yoğun koşuların üçer tekrarı ödevde isteğe bağlıdır.

11 / Trace ile nedenini göstermek

CSV sayıyı, zaman çizgisi olay sırasını açıklar

Bu pakette SystemView kaydı RAM'e alınır, deney sonunda ST-LINK Virtual COM Port üzerinden dışarı aktarılır ve .SVDat dosyası olarak açılır. SystemView'de canlı J-Link bağlantısı kurmak bu referansın yöntemi değildir.

Referans hedef NUCLEO-L476RG, bağlantı 230400 baud ve 8N1'dir. COM4 paket hazırlanırken kullanılan porttur; kendi bilgisayarında değişebilir. CubeIDE ile kaynak proje içe aktarılır, derlenir ve yüklenir; program breakpoint'te duruyorsa Resume gerekir. Ayrıntılı adımlar yerel SystemView_Kurulum.md dosyasındadır.

Kayıt sırası

1. Kayıt masasını localhost üzerinden Chrome/Edge'de aç ve doğru VCP portunu seç.
2. Varyant, yük ve trace ayarını seç; koşuyu çalıştır ve bitişi bekle.
3. CSV, ham oturum ve metadata'yı kaydet; son izi karttan alıp .SVDat indir.
4. Yeni koşudan önce dosyaları koru. Karttaki önceki iz yeni koşuda değiştirilir.
5. SystemView'de File > Open ile izi aç; Events ve Timeline'ı birlikte incele.

Bir mesajın olayları

`SEND_BEGIN` send öncesidir. `SEND_OK` veya `SEND_FAIL` API dönüşünü gösterir. `RECEIVE` consumer'ın mesajı aldığı, `DONE` servis modelinin tamamlandığı andır. Aynı run ve mesaj id değerlerini eşleştir. Farklı koşuların aynı id değerleri aynı mesaj değildir.

Pencere ve geçerlilik

RAM tamponu 32768 bayt, nominal pencere 200 ms'dir. Normal yükte ilk mesaj çevresinde, yoğun yükte 11. mesajdan önce başlar. B kuyruğunun bütün boşalması bu pencereye sığmayabilir. CSV ise bütün 44 girişimi kapsar. Trace dışında kalan bir mesaj için çalışmadı sonucu çıkarma.

Metadata'da byte sayısı, checksum, drops ve stop_reason bilgilerini kontrol et. drops pozitifse kayıt eksiktir. Stop reason 1 süreyi, 2 buffer sınırını, 0 diğer durdurmayı anlatır. DWT trace saatiyle TIM2 ölçüm saatinin mutlak değerlerini birbirinden çıkarma; olay kimliklerini eşleştir ve süreleri kendi saatleri içinde yorumla.

Üç kanıt hedefle: A/yoğun başarısız send; B/yoğun kabul ve gecikme farkı; C/yoğun Producer bloklanırken Consumer'ın ilerlemesi. Ekran görüntüsüne kimlik ve zaman aralığı ekle.

12 / Hazır kayıt nasıl yorumlanır?

Paket verisiyle bir okuma örneği

Aşağıdaki sayılar yeni ölçüm değildir. Paketteki `measurements/hardware/A_BURST_trace1_run4_1791220527624444500.json` dosyası A/yoğun, trace açık referans koşusunu tanımlar.

Kayıt alanı	Paketteki değer
run_id	4
attempted	44
accepted	24
failed	20
consumed	24
max_latency_us	60870
max_send_us	48
trace drops	0

Önce tutarlılık: 44 = 24 + 20, ayrıca servis sonunda 24 = 24. Başarısızlık oranı 20/44, yaklaşık %45,45'tir. Bu değer, kabul edilmiş mesajların sonradan kaybolduğunu söylemez; bu koşuda 20 gönderme girişimi queue'ya kabul edilmemiştir.

Maksimum generation-to-receive değeri 60870 us, yani 60,870 ms'dir. Metadata'daki max_send_us 48 us'dir; farklı bir zaman aralığını ölçer. Send'in çok kısa olması, consumer'ın mesajı çok kısa sürede aldığı anlamına gelmez.

Bu kayıt neyi tek başına kanıtlamaz?

A'daki sonuç, B'nin mutlaka daha iyi veya C'nin mutlaka kayıpsız olduğunu göstermez. Bunun için ilgili koşulların kayıtları gerekir. Drops 0 olması da trace'in bütün deneyi kapsadığı anlamına gelmez; pencere yine yaklaşık 200 ms'dir. Bir maksimum, gelecekteki bütün koşuların garantisi değildir.

Karşılaştırma cümlesinin yapısı

Ölçüm yaptıktan sonra şu yapıyı kullan: A/yoğun ile B/yoğun karşılaştırmasında yalnız kapasiteyi değiştirdim. Kabul sayısı ... değerinden ... değerine, alınan mesajların maksimum gecikmesi ... us'den ... us'ye değişti. Bu koşullarda seçimin bedeli ... oldu. Boşlukları kendi dosyalarından doldur.

Sık yapılan hatalar

- C'nin send süresini yalnız queue kopyalama maliyeti sanmak.
- Consumer'ın delay süresini CPU kullanım süresi sanmak.
- Trace penceresinde görünmeyen olayı hiç gerçekleşmedi saymak.
- B/C farkını tek değişkenin etkisi olarak açıklamak.
- Paketin hazır ölçümlerini kendi donanım koşusu olarak sunmak.

13 / Bilgini sına

Önce cevapla, sonra aşağıdaki anahtara bak

1. Kapasite 3, sıra [10,20]. Front(99), Peek ve Receive sonrası alınan değerler ve son sıra nedir?

2. Dolu queue'ya 0 tick send ne yapar? Front kullanmak sonucu değiştirir mi?

3. Producer P2, Consumer P1. Producer dolu queue'da bloklanırsa Consumer çalışabilir mi?

4. Send timeout 20 ms ise başarılı çağrı mutlaka 20 ms mi sürer?

5. Struct 42 değeriyle gönderildikten sonra yerel nesne 99 yapılırsa consumer hangi değeri alır?

6. attempted=44, accepted=30, failed=14, consumed=29. Hemen hata var diyebilir misin?

7. C'de üretimden almaya gecikme, queue'ya girmeden önce beklenen zamanı içerir mi?

8. Başarılı iki mesaj 4 ve 8 ms gecikti, bir send başarısız. Gecikme ortalaması nedir?

9. Consumer vTaskDelay ile 20 ms bekliyorsa önceliğini artırmak bu beklemeyi kaldırır mı?

10. A/B ve A/C hangi değişkenleri sınar; yedinci koşu ne içindir?

Cevap anahtarı

1. Front sonrası [99,10,20]. Peek 99 döndürür ve sırayı değiştirmez. Receive 99 döndürür; son sıra [10,20].

2. Yer açılmasını beklemeden başarısız döner. Front kapasiteyi aşmaz ve eski mesajı otomatik silmez.

3. Evet; Consumer Ready ise ve daha yüksek başka Ready task yoksa çalışabilir. Producer'ın Blocked olması buna fırsat verir.

4. Hayır. Yer varsa hemen ilerler; yer açılınca süre dolmadan da ilerleyebilir. Dönüşü scheduler geciktirebilir.

5. Değer olarak struct gönderiliyorsa 42. Pointer kuyruğunda işaret edilen nesnenin durumu ayrıca önemlidir.

6. Önce koşunun ve son servisin bitip bitmediğini kontrol et. İşlem sürüyorsa 29 normal olabilir; tamamen bitmiş koşuda fark araştırılır.

7. Evet. Bu yüzden ölçüm saf queue residence time değildir.

8. (4+8)/2 = 6 ms. Başarısızlık oranı ayrıca 1/3'tür; başarısız mesaja sıfır gecikme atanmaz.

9. Hayır. Öncelik, Blocked task'ı gecikme dolmadan çalışabilir yapmaz.

10. A/B kapasiteyi, A/C send beklemesini sınar. Yedinci koşu A/yoğun trace kapalı karşılaştırmasıdır.

Hazırlık ölçütü: Cevapları ezberlemek yerine en az bir send ve receive olayını task durumlarıyla açıklayabiliyorsan deneye geçmeye hazırsın. Zorlandığın sorunun ilgili bölümünü tekrar oku.

14 / Öğrenmeden ödeve geçiş

İlk uygulama oturumunun planı

Önce spec.md içindeki ön çalışma sorularını yanıtla. Üretici-tüketici yolunu çiz, veri kopyasını ve dolu/boş davranışını belirt. Kendi projen veya izin verilen referans firmware arasında başlangıç seçimini yap. Önceki çalışan sürümü koru ve kart/sürüm bilgisini yaz.

Sonra yalnız bir normal koşuyla bütün kayıt zincirini doğrula: derleme, kartta çalışma, sayaçlar, CSV, trace ve SystemView. Bu zincir çalışınca yedi koşu matrisini tamamla. Her koşunun dosyalarını kaydetmeden sonrakine geçme.

Analizde önce sayaç tutarlılığını, sonra sürelerin tanımını, ardından trace geçerliliğini kontrol et. A/B kapasite ve A/C bekleme karşılaştırmalarını ayrı yaz. Üç işaretli görsel, bir sayfalık yorum ve bir mesajı anlattığın 3-5 dakikalık video hazırla. Ek arayüz veya ikinci analiz aracı zorunlu değildir.

Yerel kaynak haritası

Aşağıdaki yollar hafta-03 klasörüne göredir. Paket kaynakları `kaynak/Hafta_3/` altında korunmuştur.

• `assets/ders-03.js`: Konu anlatımı ve eğitmen notlarının asıl içeriği.

• `assets/odev-03.js`: Yedi koşu, teslimler ve puanlama.

• `firmware/src/app.c`: Producer, Consumer, komutlar ve raporlama.

• `firmware/src/app_trace.c`: Trace penceresi ve olay kaydı.

• `firmware/vendor/freertos/include/queue.h`: Paket sürümünün API açıklamaları.

• `firmware/vendor/freertos/queue.c`: Queue uygulaması ve bekleme listeleri.

• `docs/SystemView_Kurulum.md`: Kart, build, bağlantı ve kayıt adımları.

• `measurements/hardware/`: Paketle gelen referans ölçümler; öğrenci ölçümü değildir.

Resmî ek okuma

Queue kopyalama ve bloklanma davranışı, resmî FreeRTOS queue açıklamasıyla kontrol edildi. Peek davranışı için resmî API sayfası ek başvurudur. Çevrimiçi kontrol: 12 Ekim 2026. Rehberin ayrıntılı deney tanımı, güncel web örnekleri yerine sağlanan paketin sabit kaynaklarına dayanır.

FreeRTOS - Queues

FreeRTOS - xQueuePeek

Bu rehberde hiçbir yeni donanım sonucu üretilmedi. Teslim tarihi pakette belirlenmediği için tarih eklenmedi. Zorunlu gereksinimleri, kabul kontrolünü ve hedef klasör düzenini spec.md üzerinden takip et.